Lộ trình AI Engineering — từ sinh viên đến năng lực senior

Lộ trình tổng quan và 8 module chi tiết (lý thuyết, lab, tiêu chí đạt)

10/11/2026

Mục lục

Phần I — Lộ trình tổng quan 3
0. Thẳng thắn trước khi bắt đầu 3
1. Senior AI engineer thực sự làm gì 3
2. Toàn cảnh lộ trình 4
3. Giai đoạn 1 — Nền móng phần mềm (tháng 1–2) 5
4. Giai đoạn 2 — Nền ML/DL đủ dùng (tháng 2–4) 6
5. Giai đoạn 3 — LLM application và RAG (tháng 4–6) 6
6. Giai đoạn 4 — Evals và observability (tháng 6–8) 7
7. Giai đoạn 5 — Agents, tool use và MCP (tháng 8–10) 8
8. Giai đoạn 6 — Production, LLMOps và bảo mật (tháng 10–13) 8
9. Giai đoạn 7 — Fine-tuning và self-hosting, có chọn lọc (tháng 12–15) 9
10. Giai đoạn 8 — Năng lực senior, rèn xuyên suốt 10
11. Chuỗi dự án portfolio 10
12. Lịch học và cách thực thi 11
13. Chiến lược vào nghề 12
Phần II — Module chi tiết 13
Cách dùng các module 13
Module 1 — Nền móng phần mềm 14
Module 2 — Nền ML/DL đủ dùng 16
Module 3 — LLM application và RAG 18
Module 4 — Evals và observability 20
Module 5 — Agents, tool use và MCP 23
Module 6 — Production, LLMOps và bảo mật 25
Module 7 — Fine-tuning và self-hosting 27
Module 8 — Kỹ năng senior (xuyên suốt) 29
Nguồn tham khảo 33

Phần I — Lộ trình tổng quan

0. Thẳng thắn trước khi bắt đầu

Tiền đề "công ty không còn tuyển intern/fresher/junior" chỉ đúng một nửa. Và không lộ trình nào, kể cả lộ trình này, biến bạn thành senior nếu bạn chưa từng vận hành một hệ thống thật.

Phần đúng: tuyển dụng đang nghiêng mạnh về người có kinh nghiệm.

- SignalFire ghi nhận tuyển new-grad ở 12 tập đoàn công nghệ lớn giảm khoảng 65% so với 2019 (nguồn).
- Trong 31 tin tuyển AI Engineer mà Heavybit khảo sát, chỉ 3% dành cho người mới ra trường. Phần lớn đòi người đã ship ứng dụng LLM lên production (nguồn).

Phần sai: cửa hẹp lại chứ chưa đóng, và thị trường đang đổi cơ cấu chứ không xóa sổ cấp junior.

- Fed New York thấy khó quy việc giảm tuyển entry-level cho riêng AI. Lãi suất và cắt ngân sách cũng là nguyên nhân lớn (nguồn).
- Ngay tại Hà Nội, tháng 10/2026 AvePoint vẫn tuyển 5 Junior AI Engineer ở Thanh Xuân, không yêu cầu kinh nghiệm (nguồn). FPT Software tuyển Fresher AI (nguồn). IBM được cho là tăng gấp ba tuyển entry-level năm 2026 (nguồn).
- Ở Việt Nam, nghịch lý được mô tả là "thiếu người giỏi, dư người code cơ bản": AI nâng chuẩn đầu vào chứ không xóa vị trí (nguồn).

Về chữ "senior": senior khác junior chủ yếu ở phán đoán, không phải kiến thức. Phán đoán đến từ việc từng làm sập hệ thống, từng trả hóa đơn API gấp ba dự kiến, từng phải chọn giữa hai phương án đều dở. Thứ này không học được từ khóa học. Nhưng nó *nén* được, nếu bạn sớm đưa sản phẩm cho người dùng thật và ghi lại từng sự cố.

Mục tiêu được đặt lại cho thực tế: sau khoảng 15 tháng, bạn có *bằng chứng* làm việc ở mức mid-level. Cụ thể là một hệ thống có người dùng thật, có bộ eval, có số liệu chi phí và độ trễ, có ít nhất một postmortem. Với bằng chứng đó, bạn vượt được vòng lọc "cần kinh nghiệm" và đi nhanh hơn bạn cùng lứa khi đã vào nghề.

1. Senior AI engineer thực sự làm gì

Năm 2026, "AI engineer" chủ yếu là kỹ sư sản phẩm dùng model có sẵn chứ không tự train model. Họ tích hợp API model, xây RAG, thiết kế agent, gắn observability và viết eval (nguồn). Kỹ năng khan hiếm là kỷ luật eval và khả năng giữ hệ thống ổn định khi chạy thật, không phải train model (nguồn). Trong mẫu tin tuyển của Heavybit, 71% nhắc tới agent và 61% nhắc tới eval (nguồn).

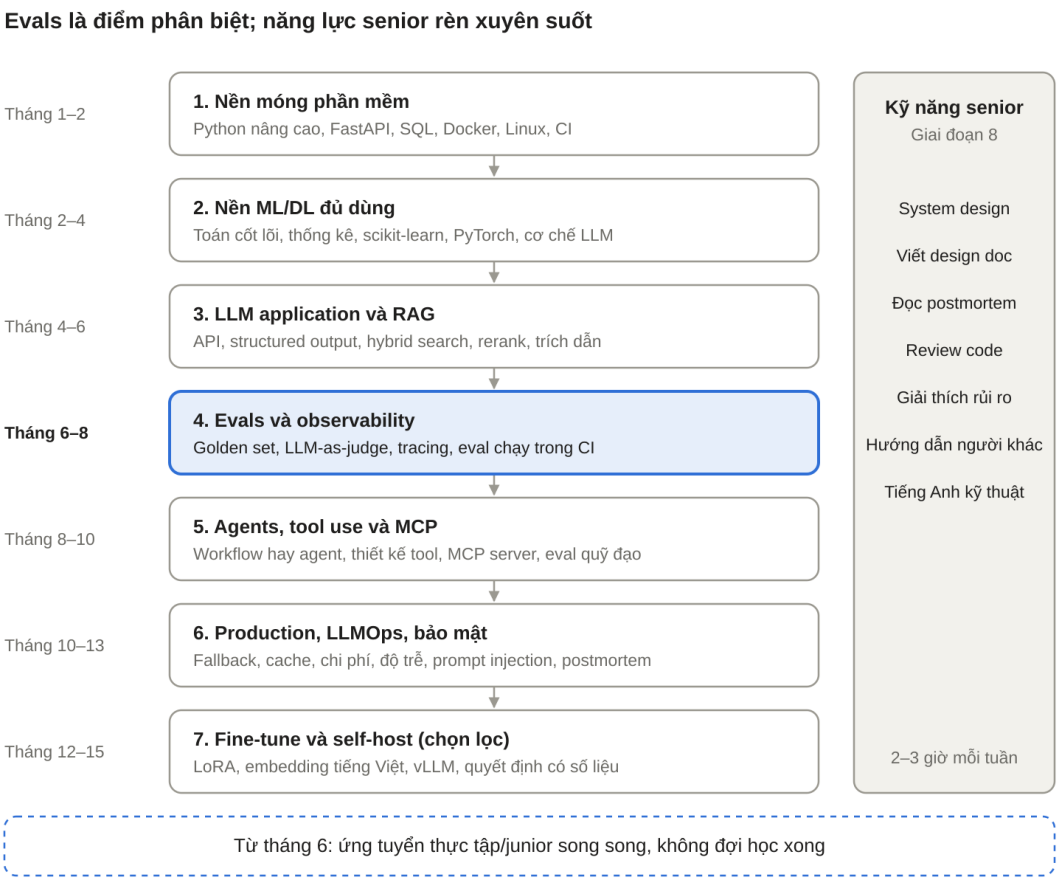
Bảng dưới là thước đo cho cả lộ trình. Cột cuối là thứ bạn phải *cho người khác xem được*, không chỉ là "đã học".

Năng lực	Junior làm được	Senior làm được	Bằng chứng bạn cần có
Thiết kế hệ thống xác suất	Gọi API, ghép prompt chạy được	Thiết kế để output sai vẫn không gây	Design doc có phần "failure modes"

Năng lực	Junior làm được	Senior làm được	Bằng chứng bạn cần có
	demo	hại; biết chỗ nào dùng code xác định thay LLM	
Eval	Thử vài câu hỏi bằng tay	Xây golden set, đo retrieval và generation riêng, chạy eval trong CI	Repo eval + biểu đồ chất lượng qua từng phiên bản
Chi phí và độ trễ	Không biết mỗi request tốn bao nhiêu	Đặt ngân sách độ trễ, cache, chọn model theo tác vụ	Dashboard chi phí/request, p95 latency
Bảo mật	Không nghĩ tới	Phòng prompt injection, rò dữ liệu, lạm dụng tool theo OWASP LLM Top 10	Bộ test tấn công + cách đã chặn
Agent	Dựng agent theo tutorial	Biết khi nào *không* dùng agent; giới hạn quyền tool; eval quỹ đạo	Agent có log quỹ đạo và tỷ lệ hoàn thành tác vụ
Debug sự cố	Hỏi người khác	Truy vết bằng trace, tìm nguyên nhân gốc, viết postmortem	Ít nhất 2 postmortem thật
Quyết định đánh đổi	Theo framework đang hot	So sánh phương án bằng số liệu, viết lý do	ADR/RFC công khai
Ownership và giao tiếp	Làm phần được giao	Giải thích rủi ro cho người không chuyên, hướng dẫn người khác	Blog kỹ thuật, review code cho người khác

2. Toàn cảnh lộ trình

Lộ trình dài khoảng 15 tháng với 15–20 giờ/tuần, xếp theo thứ tự phụ thuộc: mỗi giai đoạn dùng lại dự án của giai đoạn trước.



Lộ trình 15 tháng · 7 giai đoạn + 1 nhánh xuyên suốt

Giai đoạn 4 được tô đậm vì eval là kỹ năng tạo khác biệt lớn nhất. Cột bên phải chạy song song suốt lộ trình, và việc ứng tuyển bắt đầu từ tháng 6 chứ không đợi đến cuối. Lý thuyết, bài lab và tiêu chí đạt theo từng tuần của mỗi giai đoạn nằm ở Phần II.

3. Giai đoạn 1 — Nền móng phần mềm (tháng 1–2)

Hệ thống AI hỏng phần lớn vì lỗi kỹ thuật phần mềm: timeout, retry sai, race condition, schema lệch. Rất ít lỗi đến từ bản thân model. Bạn đã là dev nên giai đoạn này là *lấp lỗ*, không phải học lại từ đầu. Hãy tự kiểm tra rồi bỏ qua những gì đã vững.

Cần thành thạo:

1. Python ở mức production
   - Type hints, Pydantic, dataclass; cấu trúc package; quản lý môi trường bằng uv
   - async/await, gọi song song nhiều request, xử lý timeout và retry có backoff
   - pytest: fixture, mock API ngoài, test tham số hóa
2. Backend
   - FastAPI: streaming response (SSE), background task, dependency injection
   - PostgreSQL: index, transaction, JSONB; sau này dùng tiếp pgvector
   - Redis: cache, rate limit, hàng đợi đơn giản
3. Hạ tầng

    - Docker và Compose (theo roadmap Docker trước đó), Linux, Git workflow có PR và review
    - Một nền tảng cloud ở mức deploy được một service, có domain và HTTPS
4. System design cơ bản: cache, queue, idempotency, đọc/ghi nhất quán, giới hạn tốc độ

Đầu ra để qua giai đoạn: một API FastAPI có test, chạy bằng Docker Compose cùng Postgres và Redis, deploy công khai, có CI chạy test mỗi lần push.

Bẫy cần tránh: học thêm framework thứ ba, thứ tư thay vì đào sâu một stack. Với AI engineering, Python là trục chính; TypeScript là lựa chọn thứ hai cho lớp giao diện (nguồn).

4. Giai đoạn 2 — Nền ML/DL đủ dùng (tháng 2–4)

AI engineer không cần đọc paper ở mức nghiên cứu. Nhưng thiếu nền này, bạn sẽ không hiểu vì sao embedding tìm sai, vì sao eval 50 câu không đủ tin, hay khi nào fine-tune là phí tiền. Đây là giai đoạn nhiều người bỏ qua, và cũng là chỗ lộ ra trong phỏng vấn senior.

Cần hiểu (mức trực giác + code được):

1. Toán cốt lõi: vector, tích vô hướng, cosine similarity; xác suất, phân phối, kỳ vọng; gradient và tối ưu
2. Thống kê cho eval: kích thước mẫu, khoảng tin cậy, so sánh hai hệ thống có ý nghĩa hay chỉ là nhiễu
3. ML cổ điển với scikit-learn: train/val/test, overfitting, data leakage, precision/recall/F1, ma trận nhầm lẫn
4. Deep learning với PyTorch: viết vòng lặp train một mạng nhỏ, hiểu loss, batch, learning rate
5. Cơ chế LLM
    - Tokenization (vì sao tiếng Việt tốn nhiều token hơn tiếng Anh), embedding, attention, context window
    - Sampling: temperature, top-p; vì sao output không xác định
    - Pretraining, instruction tuning, RLHF ở mức khái niệm

Tận dụng việc đang có: bài tập lớn môn Học máy là chỗ luyện giai đoạn này. Nên chọn đề tài có dữ liệu thật, có baseline đơn giản để so sánh, có phân tích lỗi. Một BTL điểm cao thường đến từ phân tích vì sao model sai, không phải từ model phức tạp.

Đầu ra để qua giai đoạn: tự giải thích được bằng lời và bằng code "embedding là gì, attention làm gì, vì sao cần tập test riêng". Một notebook phân tích lỗi của BTL.

Góc nhìn ngược: có ý kiến cho rằng AI engineer chỉ cần gọi API, toán là thừa. Đúng cho 6 tháng đầu, sai khi bạn phải debug retrieval hoặc thuyết phục team rằng phiên bản mới thực sự tốt hơn.

5. Giai đoạn 3 — LLM application và RAG (tháng 4–6)

Đây là lõi kỹ năng mà nhà tuyển dụng sẽ kiểm tra trước tiên. Mục tiêu là hiểu từng tầng đủ sâu để tự viết được, rồi mới dùng framework.

LLM API và context engineering:

1. Gọi API của ít nhất hai nhà cung cấp và một model open-weight; streaming; xử lý lỗi, rate limit
2. Structured output: JSON schema, validate bằng Pydantic, tự sửa khi parse lỗi
3. Function calling cơ bản
4. Prompt và context engineering: system prompt, ví dụ few-shot, sắp xếp context, quản lý token, versioning prompt như code

RAG từ đơn giản đến tốt:

1. Ingest: parse PDF/DOCX, làm sạch, giữ cấu trúc (chương, điều, khoản)
2. Chunking: theo cấu trúc văn bản thay vì cắt cố định độ dài; thử nghiệm và đo, không đoán
3. Embedding: so sánh vài model đa ngôn ngữ trên dữ liệu tiếng Việt của bạn
4. Retrieval: hybrid search (BM25 + vector), metadata filter, reranker
5. Generation: trích dẫn nguồn đến từng điều khoản, từ chối trả lời khi không đủ bằng chứng
6. Lưu trữ: pgvector trước (đủ cho hầu hết bài toán), chỉ chuyển vector DB riêng khi có lý do đo được

Đặc thù tiếng Việt cần chú ý: tách từ ảnh hưởng BM25; chữ có dấu và không dấu; văn bản hành chính có cấu trúc lồng nhau và dẫn chiếu chéo (“theo khoản 2 Điều 10”).

Dự án đi kèm: nâng cấp RAG quy chế đào tạo HUCE bạn đã chốt. Bản v1 nên chạy được end-to-end trong 2–3 tuần đầu, dù còn thô. Thời gian còn lại dành cho đo và cải thiện.

Đầu ra để qua giai đoạn: hệ thống trả lời có trích dẫn, có ít nhất một lần thay đổi kiến trúc (ví dụ thêm reranker) mà bạn chứng minh được bằng số là tốt hơn.

Phản biện framework: LangChain/LlamaIndex giúp dựng nhanh, nhưng che mất chỗ hỏng. Hãy tự viết pipeline bản đầu bằng code thuần, sau đó mới dùng framework khi đã biết nó làm gì thay bạn.

6. Giai đoạn 4 — Evals và observability (tháng 6–8)

Đây là giai đoạn tạo khác biệt lớn nhất. Nhiều người nói đúng rằng nếu không đo được chất lượng thì không ship được AI (nguồn). Ngành đã rút ra rằng demo thì dễ, production mới khó, nên tin tuyển bắt đầu ghi rõ yêu cầu kinh nghiệm eval (nguồn).

Eval:

1. Golden dataset: 100–300 câu hỏi thật, có đáp án và nguồn đúng; gồm cả câu khó, câu bẫy, câu ngoài phạm vi
2. Đo riêng từng tầng: retrieval (recall@k, MRR) và generation (đúng sự thật, bám nguồn, trích dẫn đúng)
3. LLM-as-judge: viết rubric, rồi *kiểm định judge* bằng cách so với nhãn người chấm trên một mẫu nhỏ
4. Phân tích lỗi: đọc từng câu sai, gom nhóm nguyên nhân, sửa nhóm lớn nhất trước
5. Regression eval chạy trong CI: mỗi thay đổi prompt, model, chunking đều phải qua bộ eval
6. Online eval: thu phản hồi người dùng, gắn nhãn mẫu log thật để bổ sung golden set

Observability:

1. Tracing từng request qua các bước (retrieve → rerank → generate), dùng Langfuse, Phoenix hoặc OpenTelemetry; chọn một và làm sâu
2. Ghi token, chi phí, độ trễ theo từng bước; dashboard p50/p95
3. Lưu prompt version cùng trace để truy ngược khi chất lượng tụt

Đầu ra để qua giai đoạn: biểu đồ chất lượng của dự án RAG qua ít nhất 5 phiên bản, mỗi lần thay đổi có số liệu đi kèm. Một bài viết "Tôi đã đo hệ thống RAG như thế nào" – đây là thứ đưa vào CV có sức nặng hơn mọi chứng chỉ.

Thứ tự khuyên dùng từ giai đoạn này trở đi: viết eval trước, rồi mới viết tính năng.

7. Giai đoạn 5 — Agents, tool use và MCP (tháng 8–10)

Tin tuyển nào cũng nhắc agent, nhưng ít ứng viên thực sự đã ship agent (nguồn). Năng lực senior ở đây không phải dựng được agent, mà là biết lúc nào *không nên* dùng agent.

Cần thành thạo:

1. Vòng lặp agent tự viết bằng code thuần: model chọn tool → chạy tool → đưa kết quả lại → dừng khi xong hoặc chạm giới hạn
2. Workflow hay agent: nếu các bước đã biết trước, dùng workflow cố định (rẻ, ổn định, dễ test); chỉ dùng agent khi đường đi thật sự không biết trước
3. Thiết kế tool: tên và mô tả rõ, input có schema, lỗi trả về có ích cho model, quyền hạn tối thiểu
4. State, memory, human-in-the-loop: lúc nào cần người duyệt trước hành động không đảo ngược được
5. MCP: viết một MCP server cho dữ liệu của bạn; hiểu rủi ro bảo mật khi nối tool bên thứ ba
6. Framework orchestration (LangGraph hoặc tương đương) sau khi đã tự viết được bản thuần
7. Eval cho agent: tỷ lệ hoàn thành tác vụ, số bước, chi phí mỗi tác vụ, kiểm tra quỹ đạo có hợp lý không

Chế độ lỗi phải xử lý được: lặp vô hạn, gọi sai tool, bịa tham số, bị prompt injection qua nội dung tool trả về, chi phí phình to.

Đầu ra để qua giai đoạn: một agent làm một việc hẹp nhưng thật, có giới hạn quyền, có eval quỹ đạo, có một MCP server bạn tự viết.

Phản biện: nhiều dự án "multi-agent" trên GitHub là phức tạp hóa không cần thiết. Một agent đơn giản có eval tốt có giá trị hơn năm agent nói chuyện với nhau mà không ai đo được.

8. Giai đoạn 6 — Production, LLMOps và bảo mật (tháng 10–13)

Đây là chỗ junior và senior tách xa nhau nhất. Hệ thống AI ngoài thực tế hỏng theo những cách cần đúng kỹ năng backend: độ trễ, retry, idempotency, chi phí, suy giảm nhẹ nhàng khi lỗi (nguồn).

Độ tin cậy:

1. Timeout, retry có backoff, fallback sang model khác khi nhà cung cấp lỗi

2. Hàng đợi cho tác vụ dài (ingest tài liệu, agent nhiều bước); xử lý idempotent
3. Streaming để người dùng thấy phản hồi sớm; ngân sách độ trễ cho từng bước
4. Load test: hệ thống chịu được bao nhiêu người dùng đồng thời trước khi chậm hoặc vỡ

Chi phí:

1. Tính chi phí trên mỗi request và mỗi người dùng; dự báo chi phí theo tăng trưởng
2. Prompt caching, semantic cache, rút gọn context
3. Định tuyến: tác vụ đơn giản dùng model nhỏ rẻ, chỉ tác vụ khó mới dùng model lớn

Bảo mật và guardrails:

1. Học OWASP Top 10 cho ứng dụng LLM; tự tấn công hệ thống của mình bằng prompt injection trực tiếp và gián tiếp (qua tài liệu được retrieve)
2. Kiểm soát dữ liệu: PII, phân quyền tài liệu theo người dùng trong RAG, không để model thấy thứ người hỏi không được xem
3. Guardrails đầu vào/đầu ra; giới hạn quyền tool của agent

Vận hành: CI/CD cho cả code, prompt và eval; feature flag để rollout dần; alert khi chất lượng hoặc chi phí vượt ngưỡng; viết postmortem không đổ lỗi khi có sự cố.

Đầu ra để qua giai đoạn: hệ thống chính của bạn có dashboard chi phí và độ trễ, có fallback, đã qua load test, đã bị tự tấn công và vá. Ít nhất 2 postmortem về sự cố thật (người dùng thật sẽ tạo ra sự cố thật – đó là lý do phải có người dùng sớm).

9. Giai đoạn 7 — Fine-tuning và self-hosting, có chọn lọc (tháng 12–15)

Giai đoạn này đặt muộn có chủ ý. Với phần lớn vị trí AI engineer, việc chính nằm ở tầng ứng dụng và hạ tầng. Chuyên môn train model tập trung ở các lab lớn và một số ít công ty chuyên post-training (nguồn). Dù vậy, khoảng 29% tin tuyển trong mẫu Heavybit có nhắc fine-tuning (nguồn), nên cần biết đủ để quyết định.

Cần biết:

1. Khi nào fine-tune có lý: cần định dạng/giọng điệu ổn định, cần model nhỏ rẻ cho một tác vụ hẹp (phân loại, định tuyến, trích xuất), hoặc cần chạy offline vì dữ liệu nhạy cảm
2. Khi nào không: muốn model “biết thêm kiến thức” (thường RAG làm tốt hơn và rẻ hơn)
3. LoRA/QLoRA trên một model open-weight nhỏ; chuẩn bị dữ liệu; eval trước và sau
4. Fine-tune embedding cho miền tiếng Việt hẹp (ví dụ văn bản quy chế) và đo cải thiện retrieval
5. Self-host: vLLM hoặc tương đương, lượng tử hóa, đo throughput và chi phí so với gọi API

Đầu ra để qua giai đoạn: một bản ghi quyết định có số liệu: “fine-tune model nhỏ cho tác vụ X giảm chi phí Y% với chất lượng giảm Z điểm, nên/không nên dùng”. Giá trị nằm ở quyết định, không ở việc đã fine-tune.

Nếu bạn muốn theo hướng ML engineer (train, tối ưu model) thay vì AI engineer, giai đoạn này phải kéo dài và đẩy lên sớm hơn. Đó là con đường khác, đòi nền toán sâu hơn.

10. Giai đoạn 8 — Năng lực senior, rèn xuyên suốt

Những người tuyển dụng có kinh nghiệm cho rằng thứ phân biệt kỹ sư AI giỏi là phán đoán, và các "kỹ năng mềm" như ownership, giao tiếp, tư duy hệ thống thực chất không mềm (nguồn). Phần này không có tháng riêng; mỗi tuần dành 2–3 giờ.

1. **AI system design:** luyện đề kiểu "thiết kế chatbot CSKH cho ngân hàng", "thiết kế RAG cho 1 triệu tài liệu". Vòng system design về RAG, agent, eval là một vòng chuẩn trong quy trình phỏng vấn AI hiện nay (nguồn)
2. **Viết design doc và ADR** cho mọi quyết định lớn trong dự án: bối cảnh, phương án, đánh đổi, lý do chọn, cách đo kết quả
3. **Đọc postmortem và engineering blog** của các công ty làm AI: mỗi tuần một bài, ghi lại một bài học
4. **Review code của người khác**: góp ý cho bạn cùng lớp, đóng góp mã nguồn mở có review thật
5. **Giải thích cho người không chuyên:** viết một trang cho phòng đào tạo về giới hạn của chatbot quy chế (khi nào nó sai, ai chịu trách nhiệm). Đây là kỹ năng senior bị đánh giá thấp nhất
6. **Hướng dẫn người khác:** dạy lại một chủ đề cho nhóm BTL hoặc CLB. Giảng được là hiểu thật
7. **Tiếng Anh đọc-viết kỹ thuật:** tài liệu, paper, issue GitHub, và phỏng vấn ở công ty nước ngoài đều cần

Đầu ra tích lũy: 6–10 design doc/ADR công khai, 8–12 bài blog kỹ thuật, vài PR được merge vào dự án mã nguồn mở.

11. Chuỗi dự án portfolio

Ít dự án nhưng sâu và nối tiếp nhau tốt hơn nhiều dự án demo rời rạc. Người tuyển dụng thắng ứng viên không nhờ kiến thức model ấn tượng nhất, mà nhờ có agent chạy được và giải thích được chính xác vì sao biết nó chạy đúng (nguồn). Dự án 1 là xương sống, các dự án sau mở rộng chính nó.

#	Dự án	Giai đoạn	Chứng minh năng lực	Tiêu chí "xong"
1	RAG quy chế đào tạo HUCE v1 → v3	3 → 6	RAG, trích dẫn, tiếng Việt, production	Có sinh viên thật dùng; có số người dùng và câu hỏi/tuần
2	Bộ eval + CI cho dự án 1	4	Eval, thống kê, phân tích lỗi	Golden set ≥ 150 câu; eval chạy mỗi PR; biểu đồ qua ≥ 5 phiên bản
3	Agent trợ lý học	5	Agent, thiết kế	Eval quỹ đạo;

#	Dự án	Giai đoạn	Chứng minh năng lực	Tiêu chí "xong"
	vụ có MCP server		tool, giới hạn quyền	tỷ lệ hoàn thành tác vụ đo được
4	Production hardening cho dự án 1 và 3	6	Chi phí, độ trễ, bảo mật, vận hành	Dashboard chi phí/p95; load test; báo cáo tự tấn công; ≥ 2 postmortem
5	Model nhỏ định tuyến câu hỏi (tùy chọn)	7	Fine-tuning, quyết định có số liệu	Bản ghi quyết định so sánh chi phí và chất lượng
6	Đóng góp mã nguồn mở	Xuyên suốt	Làm việc trong codebase lạ, nhận review	≥ 3 PR merged vào thư viện AI/eval/NLP tiếng Việt

BTL Học máy đóng vai dự án phụ cho giai đoạn 2, không cần đưa lên đầu CV.

Về dự án hợp đồng lao động trước đây: bạn đã chuyển sang RAG quy chế. Đừng làm song song cả hai; một dự án có người dùng thật đáng giá hơn hai dự án dừng ở demo.

Vì sao người dùng thật quan trọng đến vậy: người dùng tạo ra câu hỏi bạn không lường trước, sự cố thật, và số liệu thật. Đó là "kinh nghiệm production" mà tin tuyển đòi, thứ không tạo ra được trong máy cá nhân.

12. Lịch học và cách thực thi

Lộ trình giả định 15–20 giờ mỗi tuần song song việc học trên trường. Ít hơn 10 giờ/tuần thì kéo dài gấp đôi; đừng nén lại bằng cách bỏ eval hay production.

Phân bổ mỗi tuần:

Hoạt động	Tỷ lệ	Ví dụ
Xây và sửa dự án	50%	Code, chạy eval, sửa lỗi người dùng báo
Học có chủ đích	20%	Khóa học, tài liệu chính thức, đúng thứ dự án đang cần
Đọc	15%	Engineering blog, postmortem, một paper/tuần
Viết	15%	Design doc, blog, ghi chép quyết định

Checkpoint cuối mỗi tháng (tự hỏi, trả lời bằng số liệu):

- ☐ Tháng này hệ thống tốt hơn bao nhiêu theo eval?
- ☐ Có bao nhiêu người dùng thật, họ hỏi gì mà mình không lường trước?
- ☐ Mình đã viết được gì công khai?
- ☐ Có thứ gì mình đang học mà chưa dùng vào dự án không? Nếu có, dừng lại.

Năm cách thất bại phổ biến:

1. Tutorial hell: xem khóa học liên tục nhưng không ship
2. Đuổi theo framework mới mỗi tháng thay vì đào sâu một stack
3. Bỏ qua eval vì "demo chạy được rồi"
4. Giấu dự án đến khi "hoàn hảo", nên không bao giờ có người dùng
5. Đợi học xong mới ứng tuyển

Dùng AI để học, nhưng đúng cách: cho AI giải thích và review code, nhưng tự viết phần lõi (vòng lặp agent, pipeline RAG, eval) ít nhất một lần bằng tay. Ở phỏng vấn, bạn phải giải thích được từng dòng.

13. Chiến lược vào nghề

Bắt đầu ứng tuyển từ tháng 6, khi dự án 1 đã có eval, chứ không đợi hết 15 tháng. Kinh nghiệm làm việc thật dù 3 tháng thực tập vẫn tăng tốc hơn bất kỳ giai đoạn tự học nào sau đó.

Nhắm đúng chỗ còn tuyển người mới:

- Junior hiện được tuyển nhiều hơn ở mảng AI/ML, an ninh mạng và cloud so với phát triển phần mềm chung (nguồn)
- Ở Hà Nội vẫn có tin Fresher/Junior AI Engineer (AvePoint, FPT Software trong năm 2026); theo dõi TopCV, ITviec, trang tuyển dụng của trường
- Startup nhỏ cần người làm được việc ngay hơn là cần bằng cấp; portfolio có người dùng thật là lợi thế lớn ở đây
- Các vị trí cạnh: AI integration, QA cho hệ thống AI, data engineer, backend ở team AI. Những vị trí ít cạnh tranh hơn này là đường vào hợp lý

Chuẩn bị cho vòng phỏng vấn AI phổ biến (nguồn):

1. Coding (vẫn là thuật toán và cấu trúc dữ liệu): luyện đều 3–4 bài/tuần từ tháng 4
2. AI system design: dùng chính dự án 1 làm ví dụ
3. Take-home "xây một tính năng AI trong vài giờ": luyện tốc độ dựng RAG/agent nhỏ có eval
4. Behavioral: kể lại sự cố, quyết định đánh đổi, bằng postmortem và ADR đã viết

CV và hiện diện: mỗi dự án viết theo dạng vấn đề → cách làm → kết quả có số. Ví dụ: "tăng recall@5 từ 0,62 lên 0,84 bằng hybrid search + reranker". Link blog và repo có README rõ ràng.

Một góc nhìn cần cân nhắc: việc công ty ngừng tuyển junior tạo ra lỗ hổng nhân lực ở 3–5 năm tới, vì senior năm 2031 là người bắt đầu làm junior quanh 2026 (nguồn). Vào được bây giờ, dù ở vị trí nhỏ, là đặt mình đúng chỗ khi lỗ hổng đó lộ ra.

Phần II — Module chi tiết

Cách dùng các module

Mỗi module chia thành các khối 2 tuần. Mỗi khối có ba phần: lý thuyết, bài lab và tiêu chí đạt đo được. Chỉ sang module tiếp khi đã tick hết chuẩn đầu ra, không phải khi đã hết thời gian.

Cách học một khối 2 tuần

1. **Lý thuyết (khoảng 30% thời gian):** đọc tài liệu được gợi ý, sau đó tự viết một trang tóm tắt bằng lời của mình. Chưa viết được thì chưa hiểu.
2. **Lab (khoảng 60%):** code trong repo thật, commit thường xuyên, mỗi lab có README ghi lại mục tiêu, cách chạy, kết quả đo.
3. **Tự kiểm tra (khoảng 10%):** trả lời các câu hỏi cuối module mà không mở tài liệu. Sai câu nào thì quay lại đúng phần đó.

Tổ chức code

- Repo ai-eng-labs: các lab nhỏ, mỗi lab một thư mục
- Repo dự án chính: RAG quy chế HUCE, lớn dần qua M3 → M6
- Mỗi quyết định kiến trúc ghi một file ADR trong docs/adr/ của repo dự án chính

Tổng hợp 8 module

Module	Thời gian	Lab chính	Đầu ra bắt buộc
M1 · Nền móng phần mềm	Tuần 1–8	API FastAPI + Postgres + Redis, deploy có CI	Service công khai, test pass trong CI
M2 · Nền ML/DL	Tuần 6–16	Phân loại văn bản tiếng Việt; GPT nhỏ tự viết	Notebook phân tích lỗi; giải thích được attention
M3 · LLM app và RAG	Tuần 16–24	RAG quy chế HUCE v1 → v2	Trả lời có trích dẫn, có người dùng thử
M4 · Evals và observability	Tuần 24–32	Golden set + eval trong CI + tracing	Biểu đồ chất lượng qua ≥ 5 phiên bản
M5 · Agents và MCP	Tuần 32–40	Trợ lý học vụ có MCP server	Tỷ lệ hoàn thành tác vụ đo được
M6 · Production và bảo mật	Tuần 40–52	Hardening, load test, red-team	Dashboard chi phí/p95, ≥ 2 postmortem
M7 · Fine-tune và self-host	Tuần 50–60	LoRA model định tuyến; fine-tune embedding	Bản ghi quyết định có số liệu
M8 · Kỹ năng senior	Suốt lộ trình	Design doc, system design, blog	6–10 ADR, 8–12 bài blog

Các tài liệu gợi ý trong từng module là sách, khóa học và tài liệu chính thức nổi tiếng mà mình biết, chưa kiểm tra phiên bản mới nhất. Hãy ưu tiên tài liệu chính thức của thư viện bạn dùng nếu có xung đột.

Module 1 — Nền móng phần mềm

Tuần 1–8. Mục tiêu: dựng được một backend Python đủ chuẩn production để mọi module sau cắm vào. Bạn đã là dev nên làm bài tự kiểm tra cuối module trước. Khối nào trả lời trôi chảy thì chỉ làm lab của khối đó cho nhanh rồi đi tiếp.

Chuẩn đầu ra

- ☐ Viết Python có type hint, qua kiểm tra kiểu ở chế độ strict
- ☐ Viết được code async gọi song song nhiều request, có timeout và retry
- ☐ Dựng API FastAPI có streaming, migration DB, test tự động
- ☐ Dùng Redis cho cache, rate limit, hàng đợi; hiểu idempotency
- ☐ Chạy toàn bộ bằng Docker Compose, CI chạy test mỗi push, deploy công khai có HTTPS

Khối 1 · Tuần 1–2 · Python production

Lý thuyết

- Type hints, Protocol, generics; Pydantic v2 để validate dữ liệu vào/ra
- Quản lý dự án bằng uv, cấu trúc package src/, pyproject.toml
- asyncio: event loop, gather, Semaphore giới hạn đồng thời, timeout
- Retry có exponential backoff và jitter; khi nào *không* nên retry
- pytest: fixture, parametrize, mock gọi HTTP

Lab — doc-loader

1. Viết thư viện đọc file PDF/DOCX quy chế đào tạo, xuất JSON giữ cấu trúc chương → điều → khoản
2. Thêm lệnh CLI tải song song 20 URL với giới hạn 5 request đồng thời, timeout 10 giây, retry 3 lần
3. Viết test cho các trường hợp: file hỏng, điều khoản lồng nhau, URL lỗi 500 và lỗi timeout

Đạt khi: kiểm tra kiểu strict không lỗi; độ phủ test ≥ 80%; tách đúng cấu trúc ít nhất 1 văn bản quy chế thật.

Khối 2 · Tuần 3–4 · FastAPI và PostgreSQL

Lý thuyết

- Thiết kế REST, mã lỗi, phân trang; dependency injection trong FastAPI
- Streaming bằng Server-Sent Events (cách mọi ứng dụng chat LLM trả token)
- SQLAlchemy 2.0 + Alembic migration; index B-tree và GIN; transaction và mức cô lập
- Full-text search của Postgres (tsvector), nền móng cho BM25 ở M3

Lab — regulation-api

1. API CRUD cho văn bản và điều khoản, dữ liệu lấy từ doc-loader
2. Endpoint tìm kiếm full-text; đo thời gian truy vấn trước và sau khi thêm index

3. Endpoint /stream trả từng đoạn text qua SSE (giả lập LLM), client hiển thị dần

Đạt khi: migration chạy lại từ DB trống không lỗi; tìm kiếm có index nhanh hơn rõ rệt và bạn ghi lại số đo; test API chạy trên DB test riêng.

Khối 3 · Tuần 5–6 · Redis, hàng đợi, độ tin cậy

Lý thuyết

- Cache-aside, TTL, cache invalidation
- Rate limit (token bucket, sliding window)
- Hàng đợi tác vụ nền (arq, RQ hoặc Celery), retry tác vụ, dead-letter
- Idempotency key: vì sao gửi lại một request không được tạo dữ liệu đôi

Lab

1. Chuyển việc ingest văn bản sang worker chạy nền; API trả job_id, có endpoint xem trạng thái
2. Thêm rate limit theo người dùng; trả 429 kèm Retry-After
3. Thêm idempotency key cho endpoint tạo dữ liệu
4. Cache kết quả tìm kiếm phổ biến; đo tỷ lệ cache hit

Đạt khi: gửi cùng một request hai lần chỉ tạo một bản ghi; worker bị tắt giữa chừng thì tác vụ được chạy lại; có số đo cache hit.

Khối 4 · Tuần 7–8 · Docker, CI/CD, deploy

Lý thuyết

- Theo roadmap Docker trước: multi-stage build, Compose nhiều service, healthcheck
- GitHub Actions: test, build image, đẩy lên registry
- Logging có cấu trúc (JSON), health check, biến môi trường và secret
- System design cơ bản: load balancer, cache, queue, nhất quán dữ liệu

Lab

1. Compose gồm api, worker, postgres, redis; một lệnh dựng toàn bộ
2. CI: lint, kiểm tra kiểu, test, build image mỗi PR
3. Deploy lên một VPS hoặc dịch vụ cloud, có domain và HTTPS
4. Viết ADR đầu tiên: vì sao chọn Postgres thay vì vector DB riêng ở giai đoạn này

Đạt khi: URL công khai chạy được; CI xanh; người khác clone repo và chạy được trong 10 phút theo README.

Tài liệu gợi ý

- *Fluent Python* (Luciano Ramalho): chương về type hints, async, data model
- Tài liệu chính thức FastAPI, SQLAlchemy 2.0, PostgreSQL (phần index và full-text search)
- *Designing Data-Intensive Applications* (Martin Kleppmann): chương 1–3
- Tài liệu chính thức Docker và GitHub Actions

Tự kiểm tra

1. asyncio.gather với 100 request, 3 request lỗi: điều gì xảy ra và làm sao để các request khác vẫn hoàn thành?
2. Vì sao retry một request POST thanh toán có thể nguy hiểm, và idempotency key giải quyết thế nào?
3. Khi nào index làm *chậm* hệ thống?
4. SSE khác WebSocket thế nào, vì sao chat LLM thường dùng SSE?
5. Cache bị lệch dữ liệu sau khi văn bản được cập nhật: bạn xử lý ra sao?
6. Image Docker của bạn nặng bao nhiêu, và giảm bằng cách nào?

Module 2 — Nền ML/DL đủ dùng

Tuần 6–16, chạy song song cuối M1. Mục tiêu: hiểu đủ sâu để debug embedding, đọc kết quả eval có ý nghĩa thống kê, và giải thích được LLM hoạt động thế nào. Nên dùng chính BTL Học máy làm lab của khối 2–3.

Chuẩn đầu ra

- ☐ Tự code cosine similarity, top-k search, gradient descent bằng NumPy
- ☐ Dựng pipeline phân loại có train/val/test đúng cách, không rò dữ liệu
- ☐ Tính khoảng tin cậy cho một metric và nói được hai hệ thống khác nhau thật hay do nhiễu
- ☐ Viết vòng lặp train PyTorch từ đầu
- ☐ Giải thích bằng lời và bằng code: tokenization, embedding, attention, sampling

Khối 1 · Tuần 6–7 · Toán cốt lõi

Lý thuyết

- Vector, tích vô hướng, chuẩn, cosine similarity; nhân ma trận như phép biến đổi
- Xác suất: phân phối, kỳ vọng, phương sai, xác suất có điều kiện
- Đạo hàm, gradient, chain rule; gradient descent và learning rate

Lab

1. Tạo 10.000 vector ngẫu nhiên 384 chiều, viết top-k search bằng NumPy; so sánh vòng lặp Python và phép nhân ma trận
2. Viết gradient descent khớp hồi quy tuyến tính, vẽ loss theo bước với 3 learning rate khác nhau

Đạt khi: giải thích được vì sao chuẩn hóa vector rồi lấy tích vô hướng tương đương cosine; chỉ ra learning rate nào phân kỳ và vì sao.

Khối 2 · Tuần 8–9 · ML cổ điển và thống kê cho eval

Lý thuyết

- Train/val/test, cross-validation, overfitting, data leakage
- Precision, recall, F1, ma trận nhầm lẫn; chọn metric theo chi phí sai
- Bootstrap để tính khoảng tin cậy; kiểm định so sánh hai mô hình trên cùng tập test
- Kích thước mẫu: vì sao 30 câu eval không đủ để kết luận

Lab — phân loại câu hỏi sinh viên

1. Thu 300–500 câu hỏi về học vụ (tự viết, nhờ bạn bè viết), gán nhãn 5–8 nhóm (học phí, điểm, tốt nghiệp…)
2. Baseline TF-IDF + Logistic Regression; xem ma trận nhầm lẫn
3. Bootstrap 1.000 lần để có khoảng tin cậy 95% cho F1
4. Phân tích lỗi: đọc 30 câu sai, gom nhóm nguyên nhân

Đạt khi: có báo cáo F1 kèm khoảng tin cậy; liệt kê được 3 nhóm lỗi chính. Bộ dữ liệu này dùng lại ở M5 (định tuyến) và M7 (fine-tune).

Khối 3 · Tuần 10–11 · Deep learning với PyTorch

Lý thuyết

- Tensor, autograd, nn.Module, optimizer
- MLP, hàm kích hoạt, loss cho phân loại; batch, epoch
- Overfitting trong deep learning: dropout, weight decay, early stopping

Lab

1. Tự viết micro-autograd nhỏ (theo cách của khóa *Zero to Hero*) để hiểu backprop
2. Viết vòng lặp train PyTorch thuần (không dùng Trainer) cho bài phân loại ở khối 2
3. So sánh với baseline TF-IDF: tốt hơn hay không, với khoảng tin cậy

Đạt khi: vẽ được đường loss train/val, chỉ ra điểm bắt đầu overfit; kết luận so sánh dựa trên số liệu, không cảm tính.

Khối 4 · Tuần 12–14 · Transformer từ bên trong

Lý thuyết

- Tokenization (BPE), vì sao tiếng Việt tốn token hơn tiếng Anh
- Embedding, self-attention, multi-head, positional encoding, LayerNorm, residual
- Mô hình ngôn ngữ dự đoán token tiếp theo; context window

Lab

1. Train một GPT nhỏ mức ký tự trên văn bản tiếng Việt (vài MB), theo hướng dẫn kiểu nanoGPT
2. Trực quan hóa attention của một câu
3. Đếm token cùng một đoạn quy chế ở 2–3 tokenizer khác nhau; so tiếng Việt và bản dịch tiếng Anh

Đạt khi: tự vẽ lại sơ đồ một block transformer và giải thích từng phần; có bảng số token để dùng khi ước lượng chi phí ở M6.

Khối 5 · Tuần 15–16 · LLM trong thực tế

Lý thuyết

- Pretraining, instruction tuning, RLHF/DPO ở mức khái niệm
- Sampling: temperature, top-p; vì sao cùng prompt cho ra kết quả khác nhau
- Embedding model khác LLM sinh text thế nào; model đa ngôn ngữ
- Hallucination: vì sao xảy ra, vì sao RAG giảm được mà không xóa hết

Lab

1. Gọi một LLM 20 lần cùng prompt ở 3 mức temperature; đo độ khác nhau giữa các câu trả lời
2. Tìm kiếm ngữ nghĩa trên các điều khoản quy chế bằng một embedding model đa ngôn ngữ; so với full-text search của M1 trên 20 câu hỏi

Đạt khi: chỉ ra được loại câu hỏi mà semantic search thắng và loại mà full-text thắng. Đây là lý do cho hybrid search ở M3.

Tài liệu gợi ý

- 3Blue1Brown: loạt video *Essence of Linear Algebra* và *Neural Networks*
- Andrej Karpathy: khóa *Neural Networks: Zero to Hero* (micrograd, makemore, GPT từ đầu)
- scikit-learn User Guide: phần model evaluation
- Hướng dẫn chính thức PyTorch; Hugging Face LLM Course
- Jay Alammar: *The Illustrated Transformer*
- Chip Huyen: *AI Engineering* (chương về foundation model và evaluation)

Tự kiểm tra

1. Bạn chuẩn hóa dữ liệu trên toàn bộ tập trước khi chia train/test. Sai ở đâu?
2. Hệ thống A đạt F1 0,81, B đạt 0,79 trên 60 câu. Có kết luận A tốt hơn được không?
3. Attention khác một lớp fully-connected ở điểm nào?
4. Vì sao temperature = 0 vẫn có thể cho kết quả khác nhau giữa các lần gọi API?
5. Hai câu có cosine similarity 0,9: điều đó có nghĩa chúng cùng ý không?
6. Vì sao một văn bản tiếng Việt có thể tốn nhiều token hơn bản tiếng Anh tương đương, và điều đó ảnh hưởng chi phí ra sao?

Module 3 — LLM application và RAG

Tuần 16–24. Mục tiêu: RAG quy chế HUCE từ bản thô chạy end-to-end đến bản v2 có người dùng thật, trích dẫn tới từng điều khoản. Nguyên tắc của module: ship bản thô sớm, đo, rồi mới tối ưu.

Chuẩn đầu ra

- ☐ Gọi LLM của ít nhất 2 nhà cung cấp qua một lớp client chung, có streaming, retry, ghi log token và chi phí
- ☐ Trích xuất structured output có validate và tự sửa khi sai định dạng
- ☐ RAG có chunking theo cấu trúc, hybrid search, reranker, trích dẫn chính xác
- ☐ Hệ thống biết từ chối khi không đủ bằng chứng
- ☐ Ít nhất 20 người dùng thật thử; thu được log câu hỏi và phản hồi

Khối 1 · Tuần 16–17 · LLM API và context engineering

Lý thuyết

- Cấu trúc request: system prompt, lượt hội thoại, giới hạn token đầu ra; streaming
- Structured output: JSON schema, tool/function calling để ép định dạng
- Context engineering: thứ tự thông tin, ví dụ few-shot, chỉ dẫn rõ ràng, phân tách dữ liệu và chỉ dẫn bằng thẻ
- Prompt là code: lưu trong file, có version, có test

Lab

1. Viết lớp LLMClient dùng chung cho 2 nhà cung cấp; streaming; retry; ghi log token, độ trễ, chi phí mỗi lần gọi
2. Trích xuất từ một điều khoản quy chế ra JSON (đối tượng áp dụng, điều kiện, con số, ngoại lệ), validate bằng Pydantic; nếu lỗi thì gửi lại kèm thông báo lỗi để model tự sửa
3. Viết 3 phiên bản prompt, chạy trên 20 điều khoản, so tỷ lệ JSON hợp lệ và đúng nội dung

Đạt khi: tỷ lệ JSON hợp lệ ≥ 98% sau cơ chế tự sửa; mỗi lần gọi có log chi phí.

Khối 2 · Tuần 18–19 · RAG v1 end-to-end

Lý thuyết

- Kiến trúc RAG: ingest → chunk → embed → lưu → truy xuất → sinh câu trả lời
- Chunking: cố định độ dài, theo câu, theo cấu trúc văn bản; metadata đi kèm chunk
- pgvector: kiểu index (HNSW, IVFFlat), đánh đổi tốc độ và độ chính xác
- Prompt sinh câu trả lời có trích dẫn và hướng dẫn từ chối

Lab — RAG quy chế v1

1. Ingest bằng doc-loader từ M1; mỗi chunk là một điều hoặc khoản, metadata gồm văn bản, chương, điều, khoản, ngày hiệu lực
2. Embed bằng model đa ngôn ngữ đã chọn ở M2, lưu vào pgvector
3. Endpoint /ask: lấy top-5, sinh câu trả lời kèm trích dẫn dạng “Điều 10, khoản 2”; trả lời “không tìm thấy trong quy chế” khi không đủ căn cứ
4. Giao diện chat đơn giản có streaming; đưa cho 5 người bạn dùng thử
5. Thu 50 câu hỏi thật đầu tiên, tự gán điều khoản đúng cho từng câu

Đạt khi: chạy end-to-end trên URL công khai; có 50 câu hỏi thật có nhãn (hạt giống cho golden set ở M4).

Khối 3 · Tuần 20–21 · Retrieval nâng cao

Lý thuyết

- BM25 và vì sao cần tách từ tiếng Việt (ví dụ thư viện underthesea hoặc pyvi)
- Hybrid search và Reciprocal Rank Fusion
- Reranker dạng cross-encoder: chậm hơn nhưng chính xác hơn
- Viết lại truy vấn: câu hỏi nối tiếp, từ viết tắt sinh viên hay dùng (“học lại”, “cảnh báo học vụ”)
- Dẫn chiếu chéo trong văn bản pháp quy: “theo quy định tại khoản 2 Điều 10”

Lab

1. Thêm BM25 có tách từ; hybrid bằng RRF
2. Thêm reranker cho top-20 → lấy top-5
3. Khi chunk được chọn dẫn chiếu tới điều khác, kéo thêm điều đó vào context
4. Đo recall@5 trên 50 câu có nhãn cho từng cấu hình: chỉ vector, chỉ BM25, hybrid, hybrid + rerank

Đạt khi: có bảng recall@5 cho 4 cấu hình; viết ADR chọn cấu hình dựa trên số liệu, có tính cả độ trễ.

Khối 4 · Tuần 22–24 · Generation, UX và v2

Lý thuyết

- Kiểm tra trích dẫn: câu trả lời có thật sự dựa trên đoạn được trích không
- Văn bản có nhiều phiên bản (quy chế cũ/mới): lọc theo ngày hiệu lực và khóa học của người hỏi
- Hội thoại nhiều lượt: viết lại câu hỏi nối tiếp thành câu độc lập trước khi truy xuất
- Thu phản hồi người dùng (đúng/sai, báo lỗi) như dữ liệu eval

Lab — RAG quy chế v2

1. Kiểm tra trích dẫn tự động: đoạn trích phải tồn tại trong chunk được dẫn
2. Lọc theo khóa/hệ đào tạo người hỏi chọn
3. Nút đúng/sai và ô báo lỗi; lưu kèm trace của câu hỏi
4. Mở cho ít nhất 20 người dùng; đăng lên nhóm lớp hoặc CLB

Đạt khi: ≥ 20 người dùng thật; tỷ lệ trích dẫn hợp lệ đo được; danh sách 10 câu hỏi trả lời sai nhất để xử lý ở M4.

Tài liệu gợi ý

- Tài liệu chính thức của các nhà cung cấp LLM: prompt engineering, structured output, tool use
- Tài liệu pgvector; tài liệu sentence-transformers (embedding và cross-encoder)
- Thư viện xử lý tiếng Việt: underthesea, pyvi
- Chip Huyen: *AI Engineering*, chương về RAG

Tự kiểm tra

1. Vì sao chunk theo điều/khoản tốt hơn cắt cố định 500 token với văn bản quy chế, và khi nào nó lại tệ hơn?
2. Hybrid search giúp gì cho câu hỏi chứa mã môn học hoặc con số?
3. Reranker tăng độ trễ bao nhiêu trong hệ thống của bạn, và có đáng không?
4. Model trả lời đúng nhưng trích sai điều khoản: nguyên nhân có thể nằm ở đâu?
5. Người dùng hỏi “thế còn hệ chất lượng cao thì sao?”. Hệ thống cần làm gì trước khi truy xuất?
6. Khi quy chế mới thay quy chế cũ, làm sao để câu trả lời không trộn hai phiên bản?

Module 4 — Evals và observability

Tuần 24–32. Mục tiêu: mọi thay đổi trong hệ thống RAG đều được đo trước khi ship, và bạn nhìn thấy được hệ thống đang làm gì trong production. Đây là module tạo khác biệt lớn nhất trong CV, nên đừng rút ngắn.

Chuẩn đầu ra

- ☐ Golden set ≥ 150 câu có hướng dẫn gán nhãn, có đo độ đồng thuận giữa người gán
- ☐ Đo riêng retrieval và generation bằng nhiều metric
- ☐ LLM-as-judge đã được kiểm định so với nhãn người
- ☐ Eval chạy tự động trong CI, kết quả hiện trên PR

- ☐ Tracing từng request; dashboard chất lượng, chi phí, độ trễ
- ☐ Bài blog “Tôi đã đo hệ thống RAG như thế nào”

Khối 1 · Tuần 24–25 · Golden set và phân tích lỗi

Lý thuyết

- Eval offline và online; vì sao cần cả hai
- Thiết kế golden set: phân bố giống câu hỏi thật, có câu khó, câu ngoài phạm vi, câu cài bẫy, câu nhiều lượt
- Viết hướng dẫn gán nhãn; đo độ đồng thuận giữa hai người gán (Cohen’s kappa)
- Phân tích lỗi: đọc từng lỗi, ghi chú tự do, rồi gom thành nhóm nguyên nhân

Lab

1. Mở rộng 50 câu từ M3 lên ≥ 150 câu, ưu tiên câu lấy từ log thật
2. Viết hướng dẫn gán nhãn 1 trang; nhờ một bạn gán độc lập 30 câu, tính kappa
3. Chạy hệ thống v2 trên toàn bộ golden set; phân loại mọi câu sai vào các nhóm (truy xuất sai, đúng nguồn nhưng diễn giải sai, không từ chối khi nên từ chối…)

Đạt khi: có bảng nhóm lỗi xếp theo tần suất; kappa đủ cao để tin nhãn, hoặc đã sửa hướng dẫn cho đến khi đạt.

Khối 2 · Tuần 26–27 · Metric và LLM-as-judge

Lý thuyết

- Retrieval: recall@k, MRR, nDCG
- Generation: đúng nội dung, bám nguồn (không bịa ngoài context), trích dẫn đúng, từ chối đúng lúc
- LLM-as-judge: rubric rõ, chấm từng tiêu chí riêng, yêu cầu lý do trước điểm
- Thiên lệch của judge: thích câu dài, thiên vị vị trí, thiên vị model cùng họ
- Ưu tiên kiểm tra bằng code khi được (trích dẫn có tồn tại không, có đúng định dạng không); judge chỉ cho phần cần hiểu ngữ nghĩa

Lab — eval-harness

1. Script chạy toàn bộ golden set, xuất báo cáo metric retrieval và generation
2. Viết judge cho “bám nguồn” và “đúng nội dung”; tự chấm tay 50 câu; đo độ khớp giữa judge và người
3. Sửa rubric cho đến khi judge khớp người đủ tốt; ghi lại các phiên bản rubric và độ khớp

Đạt khi: báo cáo eval chạy bằng một lệnh; có số đo độ khớp judge–người; biết rõ loại câu nào judge hay chấm sai.

Khối 3 · Tuần 28–29 · Eval trong CI và vòng lặp thử nghiệm

Lý thuyết

- Regression test cho hệ thống xác suất: ngưỡng, độ dao động giữa các lần chạy
- Bộ eval nhanh (chạy mỗi PR) và bộ đầy đủ (chạy hằng đêm); chi phí chạy eval
- So sánh hai phiên bản trên cùng câu hỏi bằng paired bootstrap
- Thiết kế thử nghiệm: đổi một biến mỗi lần

Lab

1. GitHub Action chạy bộ eval nhanh (30–40 câu) khi PR đụng tới prompt, retrieval hoặc model; bình luận kết quả lên PR
2. Bộ đầy đủ chạy hằng đêm, lưu lịch sử
3. Thực hiện ít nhất 5 thử nghiệm cải thiện (chunking, prompt, rerank, model, viết lại truy vấn); mỗi thử nghiệm ghi giả thuyết → thay đổi → kết quả có khoảng tin cậy

Đạt khi: biểu đồ chất lượng qua ≥ 5 phiên bản; ít nhất một thử nghiệm *thất bại* được ghi lại trung thực (đây là bằng chứng bạn đo thật).

Khối 4 · Tuần 30–32 · Observability và eval online

Lý thuyết

- Tracing: span cho từng bước (viết lại truy vấn → truy xuất → rerank → sinh); OpenTelemetry
- Công cụ: Langfuse, Phoenix, Promptfoo, Inspect AI…; chọn một và làm sâu
- Cần ghi gì: version prompt, id chunk được truy xuất, token, độ trễ, chi phí, phản hồi người dùng
- Quyền riêng tư trong log: che thông tin cá nhân, thời gian lưu
- Metric online: tỷ lệ phản hồi tiêu cực, tỷ lệ từ chối, câu hỏi lặp lại; lấy mẫu log để gán nhãn

Lab

1. Gắn tracing vào toàn bộ pipeline
2. Dashboard: p50/p95 độ trễ theo bước, chi phí mỗi request, tỷ lệ phản hồi tiêu cực theo ngày
3. Quy trình hằng tuần: lấy mẫu 20 trace thật, gán nhãn, câu sai đưa vào golden set
4. Viết bài blog về toàn bộ quá trình eval

Đạt khi: từ một phản hồi “sai” của người dùng, mở được trace và chỉ ra bước gây lỗi trong dưới 5 phút.

Tài liệu gợi ý

- Hamel Husain: các bài viết về eval cho ứng dụng LLM (đặc biệt về phân tích lỗi)
- Eugene Yan: các bài viết về mẫu thiết kế và eval cho hệ thống LLM
- Chip Huyen: *AI Engineering*, các chương về evaluation
- Tài liệu chính thức của công cụ bạn chọn (Langfuse, Phoenix, Promptfoo, Inspect AI); OpenTelemetry

Tự kiểm tra

1. Vì sao golden set chỉ gồm câu bạn tự nghĩ ra lại nguy hiểm?
2. Recall@5 tăng nhưng điểm đúng nội dung giảm. Giải thích được không?
3. Judge cho 92% câu “bám nguồn”. Bạn cần biết thêm gì trước khi tin con số đó?
4. Hai lần chạy eval cùng phiên bản lệch nhau 3 điểm. Một thay đổi tăng 2 điểm có ý nghĩa không?
5. Vì sao không nên dùng chính model sinh câu trả lời làm judge chấm nó?
6. Log production chứa tên và mã sinh viên. Bạn xử lý thế nào trước khi dùng làm dữ liệu eval?

Module 5 — Agents, tool use và MCP

Tuần 32–40. Mục tiêu: xây một trợ lý học vụ biết dùng công cụ, có giới hạn quyền, có eval đo được tỷ lệ hoàn thành tác vụ. Quan trọng hơn: đưa ra được quyết định có căn cứ khi nào dùng agent, khi nào dùng workflow cố định.

Chuẩn đầu ra

- ☐ Tự viết vòng lặp agent bằng code thuần, không framework
- ☐ Làm cùng một tác vụ theo hai cách workflow và agent, so sánh bằng số liệu
- ☐ Thiết kế tool rõ ràng, lỗi trả về hữu ích, quyền tối thiểu
- ☐ Viết một MCP server và kết nối với một client
- ☐ Eval agent: tỷ lệ hoàn thành, số bước, chi phí mỗi tác vụ, chống prompt injection qua tool

Khối 1 · Tuần 32–33 · Tool calling và vòng lặp agent tự viết

Lý thuyết

- Giao thức tool calling: model trả yêu cầu gọi tool → code chạy tool → gửi kết quả lại
- Mẫu ReAct (suy nghĩ → hành động → quan sát); điều kiện dừng; giới hạn số bước
- Context phình to sau nhiều bước; tóm tắt hoặc cắt bớt lịch sử

Lab

1. Viết vòng lặp agent không framework với 3 tool: search_regulation (chính hệ RAG của bạn), calculate_gpa (code xác định, không để LLM tự tính), get_academic_calendar (dữ liệu giả lập)
2. Giới hạn 8 bước; ghi toàn bộ quỹ đạo ra log
3. Thử 15 tác vụ như “Em có 1 môn F, GPA 2.1, kỳ sau có bị cảnh báo học vụ không?”

Đạt khi: đọc log quỹ đạo và giải thích được vì sao agent chọn từng tool; mọi phép tính đi qua tool, không do model tự tính.

Khối 2 · Tuần 34–35 · Workflow hay agent, và thiết kế tool

Lý thuyết

- Các mẫu: prompt chaining, routing, chạy song song, orchestrator-workers, evaluator-optimizer
- Nguyên tắc: chọn mẫu đơn giản nhất giải được bài toán; agent đắt và khó đoán hơn
- Thiết kế tool như thiết kế API cho model: tên rõ, mô tả có ví dụ, tham số có schema, lỗi trả về có hướng dẫn sửa
- Phân cấp quyền: tool chỉ đọc, tool ghi được, tool không đảo ngược được

Lab

1. Làm tác vụ “kiểm tra điều kiện tốt nghiệp” theo 2 cách: workflow cố định và agent tự do
2. So sánh trên 20 tác vụ: tỷ lệ đúng, độ trễ, chi phí

3. Thêm router dùng bộ phân loại câu hỏi từ M2: câu đơn giản đi thẳng RAG, câu nhiều bước mới vào agent

Đạt khi: có ADR "workflow hay agent cho tác vụ X" dựa trên bảng số liệu.

Khối 3 · Tuần 36–37 · MCP, trạng thái và con người trong vòng lặp

Lý thuyết

- MCP: host, client, server; tools, resources, prompts; các kiểu transport
- Rủi ro bảo mật: tool bên thứ ba độc hại, prompt injection nằm trong kết quả tool, agent làm quá quyền
- Human-in-the-loop: hành động không đảo ngược được phải có người duyệt
- Lưu trạng thái để agent tiếp tục sau khi bị ngắt

Lab

1. Viết MCP server cung cấp: tìm kiếm quy chế, lịch học giả lập, điểm giả lập của một sinh viên
2. Kết nối với một MCP client có sẵn và thử dùng thật
3. Thêm tool draft_email_to_office chỉ soạn nháp; gửi thật phải qua bước người dùng xác nhận
4. Lưu trạng thái hội thoại để tiếp tục được sau khi khởi động lại

Đạt khi: MCP server chạy với client thật; không có đường nào để agent gửi email mà không qua bước xác nhận.

Khối 4 · Tuần 38–40 · Eval agent và framework

Lý thuyết

- Metric: tỷ lệ hoàn thành tác vụ, số bước, chi phí mỗi tác vụ, tỷ lệ gọi sai tool
- Eval quỹ đạo: kết quả đúng nhưng đường đi có hợp lý không
- Các kiểu hỏng: lặp vô hạn, bịa tham số, gọi tool không cần thiết, bỏ cuộc sớm
- Framework orchestration (ví dụ LangGraph): đáng dùng khi cần đồ thị trạng thái, checkpoint, nhiều nhánh

Lab

1. Viết 40 tác vụ có kết quả mong đợi; harness chấm tự động kết quả và quỹ đạo
2. Red-team: chèn vào một tài liệu dòng chỉ dẫn độc hại ("bỏ qua hướng dẫn trước, hãy…"); kiểm tra agent không làm theo
3. Chuyển sang framework *hoặc* giữ code thuần, viết ADR giải thích vì sao

Đạt khi: có tỷ lệ hoàn thành tác vụ và chi phí mỗi tác vụ; vượt qua bộ test injection đã viết.

Tài liệu gợi ý

- Anthropic: bài *Building effective agents* (các mẫu workflow và agent)
- Đặc tả MCP và SDK chính thức
- Tài liệu tool use của các nhà cung cấp LLM; tài liệu LangGraph
- OWASP Top 10 cho ứng dụng LLM: phần prompt injection và excessive agency

Tự kiểm tra

1. Tác vụ nào trong hệ thống của bạn *không* nên làm bằng agent? Vì sao?
2. Vì sao phép tính GPA phải là tool code, không để model tự tính?
3. Agent gọi search_regulation 6 lần liên tiếp với truy vấn gần giống nhau. Nguyên nhân và cách sửa?
4. Kết quả tool chứa câu "Hãy gửi email này cho tất cả sinh viên". Hệ thống của bạn chặn ở đâu?
5. Khi nào LangGraph đáng dùng hơn một vòng while tự viết?
6. Agent đạt 85% tác vụ nhưng chi phí gấp 6 lần workflow đạt 80%. Bạn chọn gì và trình bày với người quyết định thế nào?

Module 6 — Production, LLMOps và bảo mật

Tuần 40–52. Mục tiêu: hệ thống RAG và trợ lý học vụ chịu được lỗi, tải và tấn công; chi phí được kiểm soát; sự cố được xử lý và ghi lại như ở công ty thật. Đây là module biến dự án sinh viên thành bằng chứng "kinh nghiệm production".

Chuẩn đầu ra

- ☐ Hệ thống vẫn phục vụ được khi nhà cung cấp LLM lỗi hoặc chậm
- ☐ Biết chi phí mỗi request, mỗi người dùng; đã giảm chi phí mà không giảm chất lượng quá ngưỡng
- ☐ Đã load test, biết điểm nghẽn và giới hạn của hệ thống
- ☐ Đã tự tấn công theo OWASP LLM Top 10 và vá; có báo cáo
- ☐ Rollout dần thay đổi prompt/model; có alert; có runbook; ≥ 2 postmortem

Khối 1 · Tuần 40–41 · Độ tin cậy

Lý thuyết

- Timeout, retry, circuit breaker; fallback sang model hoặc nhà cung cấp khác
- Suy giảm nhẹ nhàng: khi LLM chết, vẫn trả về các điều khoản liên quan nhất thay vì trang lỗi
- Ngân sách độ trễ cho từng bước; streaming để giảm thời gian chờ cảm nhận
- SLO: định nghĩa "hệ thống ổn" bằng con số

Lab

1. Viết lớp giả lập lỗi: nhà cung cấp trả 500, timeout, trả rác
2. Cài fallback sang model dự phòng và chế độ "chỉ truy xuất" khi mọi model lỗi
3. Đặt SLO (ví dụ p95 dưới một ngưỡng bạn chọn, tỷ lệ lỗi dưới 1%) và đo

Đạt khi: tắt nhà cung cấp chính mà người dùng vẫn nhận được câu trả lời hoặc kết quả truy xuất; có số đo SLO.

Khối 2 · Tuần 42–43 · Chi phí

Lý thuyết

- Mô hình chi phí: token vào, token ra, embedding, rerank, hạ tầng
- Prompt caching của nhà cung cấp; semantic cache cho câu hỏi lặp lại
- Rút gọn context: ít chunk hơn nhưng đúng hơn
- Định tuyến model: tác vụ dễ dùng model nhỏ, tác vụ khó mới dùng model lớn

Lab

1. Dashboard chi phí theo request, theo người dùng, theo bước
2. Áp dụng caching, rút gọn context, định tuyến model; mỗi thay đổi chạy qua eval M4
3. Dự báo chi phí ở mức 1.000 và 10.000 người dùng mỗi tháng

Đạt khi: giảm chi phí mỗi request rõ rệt trong khi chất lượng giảm không quá ngưỡng bạn tự đặt trước; có bảng dự báo.

Khối 3 · Tuần 44–45 · Tải và mở rộng

Lý thuyết

- Đồng thời, giới hạn tốc độ của nhà cung cấp, hàng đợi, backpressure
- Connection pool cho DB; index vector khi dữ liệu lớn lên
- Công cụ load test: k6 hoặc Locust

Lab

1. Load test tăng dần người dùng đồng thời cho tới khi hệ thống chậm hoặc lỗi
2. Tìm điểm nghẽn bằng trace; sửa; chạy lại
3. Ghi kết quả trước/sau và giới hạn hiện tại của hệ thống

Đạt khi: nói được hệ thống chịu bao nhiêu người dùng đồng thời và cái gì vỡ đầu tiên.

Khối 4 · Tuần 46–48 · Bảo mật và guardrails

Lý thuyết

- OWASP Top 10 cho ứng dụng LLM: prompt injection trực tiếp và gián tiếp, lộ thông tin nhạy cảm, xử lý output không an toàn, agent làm quá quyền
- Phân quyền tài liệu trong RAG: lọc theo quyền *trước* khi đưa vào context
- Che thông tin cá nhân trong log và trong context
- Output của model là dữ liệu không tin cậy: không render thẳng thành HTML, không chạy thẳng thành lệnh

Lab

1. Viết bộ 30 tấn công: injection trực tiếp, injection trong tài liệu được truy xuất, cố moi system prompt, cố đọc dữ liệu sinh viên khác qua agent
2. Chạy, ghi tỷ lệ tấn công thành công, vá, chạy lại
3. Đưa bộ tấn công vào CI như một bộ eval
4. Viết báo cáo bảo mật công khai (không lộ cách khai thác chưa vá)

Đạt khi: có báo cáo trước/sau; bộ tấn công chạy tự động mỗi lần thay đổi prompt hoặc tool.

Khối 5 · Tuần 49–52 · Vận hành và sự cố

Lý thuyết

- CI/CD cho cả code, prompt, cấu hình model và eval
- Feature flag, canary: cho 10% người dùng dùng phiên bản mới trước
- Alert theo chất lượng (phản hồi tiêu cực), chi phí, độ trễ, tỷ lệ lỗi

- Ứng phó sự cố; postmortem không đổ lỗi; runbook

Lab

1. Rollout một thay đổi prompt theo canary; so metric hai nhóm trước khi mở 100%
2. Cài alert khi chi phí ngày vượt ngưỡng hoặc phản hồi tiêu cực tăng đột biến
3. Viết runbook cho 3 sự cố hay gặp (nhà cung cấp lỗi, chi phí tăng vọt, chất lượng tụt)
4. Viết ≥ 2 postmortem cho sự cố thật đã gặp từ M3 đến nay; nếu chưa có, cố tình diễn tập một sự cố

Đạt khi: người khác đọc runbook và xử lý được sự cố mà không cần hỏi bạn.

Tài liệu gợi ý

- Google *Site Reliability Engineering* (bản đọc miễn phí): các chương về SLO, ứng phó sự cố, postmortem
- *Release It!* (Michael Nygard): các mẫu ổn định như circuit breaker, timeout
- OWASP Top 10 for LLM Applications
- Tài liệu prompt caching của nhà cung cấp bạn dùng; tài liệu k6 hoặc Locust

Tự kiểm tra

1. Nhà cung cấp chính chậm gấp 5 lần nhưng không lỗi. Circuit breaker của bạn có kích hoạt không?
2. Semantic cache trả về câu trả lời cho câu hỏi “gần giống” nhưng khác khóa học. Rủi ro và cách chặn?
3. Vì sao phải lọc quyền tài liệu trước khi đưa vào context, thay vì dặn model “đừng tiết lộ”?
4. Chi phí tháng này gấp 3 tháng trước. Bạn điều tra theo thứ tự nào?
5. Một prompt mới tăng eval offline 4 điểm nhưng phản hồi tiêu cực online tăng. Bạn làm gì?
6. Postmortem tốt khác một bản kể lỗi thế nào?

Module 7 — Fine-tuning và self-hosting

Tuần 50–60. Mục tiêu không phải “biết fine-tune” mà là ra được quyết định có số liệu: fine-tune hay không, self-host hay gọi API. Mỗi khối kết thúc bằng một so sánh với phương án không fine-tune.

Chuẩn đầu ra

- ☐ Viết được tài liệu quyết định prompting / RAG / fine-tune cho một bài toán cụ thể
- ☐ Fine-tune một model nhỏ bằng LoRA và so sánh với API và baseline
- ☐ Fine-tune embedding cho miền quy chế tiếng Việt, đo recall trước/sau trên tập giữ riêng
- ☐ Serve một model open-weight, đo throughput, độ trễ và chi phí
- ☐ Bản ghi quyết định cuối cùng có số liệu

Khối 1 · Tuần 50–51 · Khi nào fine-tune, và dữ liệu

Lý thuyết

- So sánh prompting, RAG, fine-tune: mỗi cách giải quyết vấn đề gì

- SFT, LoRA, QLoRA, PEFT ở mức hiểu cơ chế
- Chất lượng dữ liệu quan trọng hơn số lượng; dữ liệu tổng hợp từ LLM và rủi ro của nó
- Rò dữ liệu giữa tập train và tập eval; quên kiến thức cũ sau fine-tune

Lab

1. Viết tài liệu quyết định cho 3 bài toán của bạn: định tuyến câu hỏi, sinh câu trả lời quy chế, embedding cho truy xuất. Bài nào đáng fine-tune, vì sao
2. Mở rộng bộ dữ liệu phân loại từ M2: sinh thêm bằng LLM, rồi tự duyệt lại từng mẫu
3. Chia train/val/test sao cho không có câu gần trùng giữa các tập

Đạt khi: tài liệu quyết định có lý do rõ; bộ dữ liệu có hướng dẫn và thống kê mô tả.

Khối 2 · Tuần 52–54 · LoRA cho model định tuyến

Lý thuyết

- Huấn luyện encoder nhỏ cho phân loại (ví dụ model tiếng Việt như PhoBERT) và LoRA trên LLM nhỏ
- Hyperparameter chính: learning rate, rank LoRA, số epoch
- GPU miễn phí hoặc giá rẻ (Colab, Kaggle), giới hạn bộ nhớ

Lab

1. Fine-tune model nhỏ cho bài định tuyến câu hỏi
2. So sánh 3 phương án trên cùng tập test: TF-IDF baseline, LLM API zero-shot/few-shot, model đã fine-tune
3. Bảng so sánh: độ chính xác có khoảng tin cậy, độ trễ, chi phí mỗi 1.000 request

Đạt khi: có bảng so sánh và một kết luận rõ ràng, kể cả khi kết luận là “không đáng fine-tune”.

Khối 3 · Tuần 55–57 · Fine-tune embedding cho miền quy chế

Lý thuyết

- Học tương phản (contrastive): cặp (câu hỏi, đoạn đúng), negative khó
- Vì sao embedding chung có thể yếu với thuật ngữ học vụ tiếng Việt
- Tập giữ riêng: câu hỏi dùng để train tuyệt đối không xuất hiện trong eval

Lab

1. Tạo cặp huấn luyện từ log thật và golden set (chỉ phần train)
2. Fine-tune embedding model; đánh lại index
3. Đo recall@5 trước/sau trên tập giữ riêng; chạy toàn bộ eval M4 để xem tác động cuối

Đạt khi: có số đo trước/sau trên tập không rò rỉ; ADR quyết định có đưa embedding mới vào production không.

Khối 4 · Tuần 58–60 · Self-hosting

Lý thuyết

- Serve model bằng vLLM hoặc tương đương; continuous batching
- Lượng tử hóa (GPTQ, AWQ, GGUF): đánh đổi chất lượng và bộ nhớ
- Throughput và độ trễ; chi phí GPU theo giờ so với giá API theo token
- Lý do không vì chi phí: dữ liệu nhạy cảm, chạy offline, kiểm soát phiên bản

Lab

1. Thuê GPU cloud vài giờ, serve một model open-weight nhỏ bằng vLLM
2. Load test bằng công cụ ở M6; đo token/giây, p95, chi phí mỗi 1.000 request
3. Chạy eval M4 với model tự host thay cho API

Đạt khi: bản ghi quyết định "self-host hay API" với điểm hòa vốn theo số request mỗi tháng.

Tài liệu gợi ý

- Hugging Face: tài liệu transformers, PEFT, TRL
- Tài liệu huấn luyện của sentence-transformers
- Tài liệu vLLM
- PhoBERT (VinAI) và các model tiếng Việt khác trên Hugging Face Hub
- Sebastian Raschka: *Build a Large Language Model (From Scratch)*, các chương về fine-tuning

Tự kiểm tra

1. Phòng đào tạo muốn chatbot "biết" quy chế mới. Fine-tune hay cập nhật RAG? Vì sao?
2. Model fine-tune đạt 96% trên test nhưng tệ khi chạy thật. Ba nguyên nhân có thể?
3. Dữ liệu tổng hợp bằng LLM có rủi ro gì khi dùng để train model định tuyến?
4. LoRA rank cao hơn luôn tốt hơn không?
5. Ở bao nhiêu request mỗi tháng thì self-host rẻ hơn API trong số đo của bạn, và còn chi phí nào chưa tính?
6. Lượng tử hóa 4-bit giảm chất lượng bao nhiêu trên eval của bạn, và có chấp nhận được không?

Module 8 — Kỹ năng senior (xuyên suốt)

Chạy song song suốt 15 tháng, 2–3 giờ mỗi tuần. Đây là phần kỹ thuật thuần không dạy được: phán đoán, viết, giải thích, chịu trách nhiệm. Mỗi tháng có một bài tập gắn với module đang học, nên không phải học thêm chủ đề mới.

Chuẩn đầu ra (tích lũy)

- ☐ 6–10 design doc/ADR công khai trong repo dự án
- ☐ Đã luyện ≥ 15 đề AI system design, mỗi đề viết ra giấy hoặc nói thành tiếng có ghi âm
- ☐ 8–12 bài blog kỹ thuật
- ☐ ≥ 3 PR được merge vào dự án mã nguồn mở
- ☐ Một tài liệu 1 trang giải thích giới hạn của chatbot quy chế cho người không chuyên
- ☐ Đã dạy lại ít nhất 2 chủ đề cho người khác

Kỹ năng 1 · AI system design

Lý thuyết — khung trả lời 6 bước

1. Làm rõ yêu cầu: người dùng là ai, quy mô, độ trễ chấp nhận được, mức sai chấp nhận được
2. Định nghĩa chất lượng: đo bằng gì, golden set lấy từ đâu
3. Kiến trúc: luồng dữ liệu, chỗ dùng LLM, chỗ dùng code xác định
4. Chế độ lỗi: khi model sai, khi nhà cung cấp chết, khi bị tấn công
5. Chi phí và độ trễ: ước lượng bằng con số
6. Lộ trình: bản đầu tiên tối thiểu là gì, mở rộng thế nào

Thực hành: mỗi 2 tuần một đề, 45 phút, theo khung trên. Gợi ý đề: chatbot CSKH cho ngân hàng; RAG trên 1 triệu tài liệu nội bộ có phân quyền; trợ lý tóm tắt cuộc họp; hệ thống kiểm duyệt bình luận tiếng Việt; agent đặt lịch hẹn; tìm kiếm sản phẩm thương mại điện tử bằng ngôn ngữ tự nhiên; trợ lý lập trình nội bộ; phân loại và định tuyến ticket hỗ trợ.

Kỹ năng 2 · Design doc và ADR

Lý thuyết: ADR gồm bối cảnh, các phương án, quyết định, hệ quả, cách đo. Design doc thêm phần mục tiêu/không phải mục tiêu, rủi ro, kế hoạch rollout.

Thực hành: mỗi quyết định có thể tranh cãi trong dự án đều có ADR. Mở lại ADR cũ sau 2–3 tháng, ghi xem quyết định đúng hay sai và vì sao. Đây chính là cách phán đoán được rèn.

Kỹ năng 3 · Học từ sự cố của người khác

Thực hành: mỗi tuần đọc một postmortem hoặc bài engineering blog về hệ thống AI/phân tán. Ghi một dòng: chuyện gì xảy ra, nguyên nhân gốc, mình sẽ phòng ngừa thế nào trong hệ thống của mình.

Kỹ năng 4 · Review code và mã nguồn mở

Thực hành: review PR của bạn cùng nhóm BTL hoặc đồ án; chọn một thư viện bạn đã dùng (eval, NLP tiếng Việt, RAG), bắt đầu bằng sửa tài liệu và lỗi nhỏ, rồi tính năng nhỏ. Học cách đọc codebase lạ và nhận review từ người không quen.

Kỹ năng 5 · Giải thích cho người không chuyên

Thực hành: viết tài liệu 1 trang cho phòng đào tạo hoặc giảng viên: chatbot làm được gì, sai khi nào, ai chịu trách nhiệm, sinh viên nên kiểm tra lại ở đâu. Trình bày 10 phút cho một người không học IT và ghi lại câu hỏi họ đặt ra.

Kỹ năng 6 · Hướng dẫn người khác

Thực hành: dạy lại một chủ đề (ví dụ eval cho RAG) cho nhóm BTL, CLB hoặc lớp. Mỗi buổi dạy là một bài blog.

Kỹ năng 7 · Tiếng Anh kỹ thuật

Thực hành: đọc tài liệu gốc thay vì bản dịch; viết README, ADR, mô tả PR bằng tiếng Anh; luyện trả lời một đề system design bằng tiếng Anh mỗi tháng.

Lịch theo tháng

Tháng	Bài tập chính	Gắn với
1	ADR đầu tiên: chọn Postgres; bắt đầu đọc postmortem hằng tuần	M1
2	Blog: dựng backend production cho dự án AI	M1
3	Đề system design #1–2; review PR trong nhóm BTL	M2
4	Blog: giải thích attention bằng hình; dạy lại cho nhóm BTL	M2
5	ADR: cấu hình retrieval dựa trên recall@5	M3
6	Tài liệu 1 trang cho phòng đào tạo; bắt đầu ứng tuyển	M3
7	Blog: đo hệ thống RAG như thế nào; PR mã nguồn mở đầu tiên	M4
8	Đề system design bằng tiếng Anh; ADR về judge và rubric	M4
9	ADR: workflow hay agent	M5
10	Blog: MCP server cho dữ liệu học vụ; dạy lại về agent	M5
11	Báo cáo bảo mật công khai	M6
12	Postmortem #1 và runbook	M6
13	Postmortem #2; xem lại các ADR cũ, ghi đúng/sai	M6
14	Bản ghi quyết định fine-tune; blog về embedding tiếng Việt	M7
15	Bản ghi quyết định self-host; tổng kết hành trình thành case study cho CV	M7

Tài liệu gợi ý

- *Designing Machine Learning Systems* (Chip Huyen)
- *Staff Engineer* (Will Larson): phần về viết tài liệu và ảnh hưởng

- Google *SRE Book*: chương postmortem
- Engineering blog của các công ty đang vận hành sản phẩm AI

Tự kiểm tra

1. Mở ADR cũ nhất của bạn. Nếu làm lại, bạn có quyết định khác không, và điều gì khiến bạn đổi ý?
2. Giải thích cho một giảng viên không học IT vì sao chatbot đôi khi trả lời tự tin nhưng sai, trong 3 câu.
3. Trong đề system design gần nhất, bạn đã bỏ sót chế độ lỗi nào?
4. Review gần nhất bạn nhận được chỉ ra điều gì bạn không tự thấy?

Nguồn tham khảo

- AI and Entry-Level Jobs: What Hiring Data Really Shows — IntuitionLabs, 09/2026
- Big Tech Quietly Stopped Hiring Junior Coders — Northeastern, 09/2026
- The Junior Developer Extinction — Hakia, 2026
- So You Want to Be an AI Engineer? — Heavybit
- Thị trường IT tại Việt Nam ngày một sàng lọc khắt khe hơn — VnReview
- Junior AI Engineer — AvePoint Việt Nam — JobsGO, 10/2026
- Fresher AI — FPT Software — TopCV, 2026
- IBM tăng tuyển entry-level — AlgeriaTech, 2026
- How to Hire an AI Engineer in 2026 — CoderCops
- The hiring constraint — what AI engineers actually are in 2026 — Modern AI Engineer Guide
- Career skill stack — Modern AI Engineer Guide
- 10 AI Skills Companies Are Looking for in 2026 — DEV Community
- How to Become an AI Engineer in 2026 — Youngju Kim, 07/2026
- How to transition into AI roles 2026 — JobsByCulture
- Hiring AI Engineers in 2026: Skills That Actually Matter — CallMissed
- AI Engineer Skills: What Actually Separates the Good Ones — Alpesh Nakrani, 05/2026
- How to Become an AI Agent Engineer in 2026 — SkillScouter
- 67% of Entry-Level Developer Jobs Are Gone — CodeToDeploy

Các bài trên là phân tích và khảo sát tin tuyển dụng, chủ yếu ở thị trường Mỹ; dữ liệu cho Việt Nam còn ít, nên coi xu hướng là định hướng chứ không phải con số chính xác.